

Chapter 63: Infinite Scroll

Scenario 1: Building infinite scroll from scratch with IntersectionObserver

Situation: Your team is replacing a "Load More" button with a seamless infinite scroll experience for a product catalog. The lead wants a clean, standalone, signal-based implementation without any third-party scroll library, and it must stop observing once there is no more data.

Question: How would you implement infinite scroll using `IntersectionObserver` in a modern standalone Angular component, and what precautions do you take around observer lifecycle?

Answer: The cleanest approach is a sentinel element placed at the bottom of the list. An `IntersectionObserver` watches it; when it intersects the viewport, you trigger the next page fetch. Use `inject()`, signals for state, and clean up the observer in `ngOnDestroy` (or via `DestroyRef`) to avoid leaks. Disconnect once `hasMore` is false so you don't keep observing dead space.

```
import { Component, ElementRef, inject, signal, viewChild, effect, DestroyRef } from
 '@angular/core';
import { ProductService } from '../product.service';

@Component({
  selector: 'app-product-list',
  standalone: true,
  template: `
    @for (item of items(); track item.id) {
      <div class="card">{{ item.name }}</div>
    }
    <div #sentinel class="sentinel"></div>
    @if (loading()) { <p>Loading...</p> }
    @if (!hasMore()) { <p>No more products.</p> }
  `,
})
export class ProductListComponent {
  private productService = inject(ProductService);
  private destroyRef = inject(DestroyRef);
  sentinel = viewChild.required<ElementRef<HTMLDivElement>>('sentinel');

  items = signal<Product[]>([]);
  page = signal(0);
  loading = signal(false);
  hasMore = signal(true);

  private observer?: IntersectionObserver;

  constructor() {
    effect((onCleanup) => {
      const el = this.sentinel().nativeElement;
      this.observer = new IntersectionObserver(
        (entries) => {
```

```

        if (entries[0].isIntersecting && !this.loading() && this.hasMore()) {
            this.loadNextPage();
        }
    },
    { root: null, rootMargin: '200px', threshold: 0 }
);
this.observer.observe(e1);
onCleanup(() => this.observer?.disconnect());
});

this.loadNextPage();
this.destroyRef.onDestroy(() => this.observer?.disconnect());
}

private loadNextPage() {
    this.loading.set(true);
    this.productService.getPage(this.page()).subscribe({
        next: (res) => {
            this.items.update((cur) => [...cur, ...res.items]);
            this.hasMore.set(res.hasMore);
            this.page.update((p) => p + 1);
            this.loading.set(false);
        },
        error: () => this.loading.set(false),
    });
}
}

```

Key tradeoffs: `rootMargin: '200px'` pre-fetches before the sentinel is fully visible, giving a smoother experience than waiting for it to hit 0px. Disconnecting the observer when `hasMore()` becomes false (or on destroy) prevents wasted callbacks and memory leaks. Using an `effect` with `onCleanup` ties observer lifecycle to the signal-driven view rather than manual `ngAfterViewInit` / `ngOnDestroy` pairs, keeping code centralized.

Interviewer intent: Tests whether the candidate can wire `IntersectionObserver` correctly with Angular's reactivity model (signals, effects, `DestroyRef`) instead of stale `ngOnInit` / `ngOnDestroy` patterns, and whether they think about cleanup.

Scenario 2: Duplicate items from race conditions during fast scrolling

Situation: QA reports that scrolling very fast on a mobile device sometimes causes the same batch of items to appear twice in the feed. The team suspects the `IntersectionObserver` callback firing multiple times before the previous request resolves.

Question: What causes duplicate items during fast scrolling, and how do you make the fetch logic race-condition-safe?

Answer: The root cause is almost always that the "am I already loading" guard is checked but not atomically set before the async operation starts, or that multiple intersection events queue up while a request is in flight (e.g., the sentinel intersects, then leaves and re-enters as content shifts). A common failure mode: `loading` is set to `true` inside the `.subscribe()` callback instead of synchronously before firing the request, leaving a window where two triggers slip through.

The fix is to guard synchronously and additionally deduplicate by request/page identity, and to cancel superseded requests with `switchMap`-like semantics if using RxJS, or a manual "current request token" if using plain signals + fetch.

```
import { Subject } from 'rxjs';
import { switchMap, exhaustMap, tap } from 'rxjs/operators';

export class FeedComponent {
  private loadTrigger$ = new Subject<void>();
  private nextPage = signal(0);
  items = signal<Item[]>([]);
  loading = signal(false);
  hasMore = signal(true);

  constructor(private feedService: FeedService) {
    this.loadTrigger$
      .pipe(
        // exhaustMap ignores new triggers while a request is in-flight -
        // exactly what we want for "don't double-fire on fast scroll"
        exhaustMap(() => {
          if (!this.hasMore()) return EMPTY;
          this.loading.set(true);
          const page = this.nextPage();
          return this.feedService.getPage(page).pipe(
            tap((res) => {
              // Dedupe defensively by id in case of overlapping pages
              this.items.update((cur) => {
                const seen = new Set(cur.map((i) => i.id));
                const fresh = res.items.filter((i) => !seen.has(i.id));
                return [...cur, ...fresh];
              });
              this.hasMore.set(res.hasMore);
              this.nextPage.set(page + 1);
              this.loading.set(false);
            })
          );
        })
      )
      .subscribe();
  }

  onSentinelIntersect() {
    this.loadTrigger$.next();
  }
}
```

`exhaustMap` is the critical operator choice: it drops any new intersection events that arrive while a page request is still in flight, rather than queuing them (which `concatMap` would do, causing eventual duplicate-looking bursts) or canceling the current one (which `switchMap` would do, wrongly aborting valid in-flight pagination). The `Set`-based id dedupe is a second line of defense against server-side overlap (e.g., items shifting between pages if new items are inserted concurrently).

Interviewer intent: Probes understanding of RxJS flattening operators and the specific reason `exhaustMap` — not `switchMap` or `mergeMap` — is correct for pagination triggers, plus defensive dedup thinking.

Scenario 3: Preserving scroll position when navigating back

Situation: Users scroll deep into an infinite list of articles, click one to read it, then press the browser back button — and land back at the top of the list, losing their place. Product wants scroll position restored exactly.

Question: How do you preserve and restore scroll position (and the already-loaded data) when a user navigates away from and back to an infinite-scroll list?

Answer: Two things must survive navigation: the loaded data (so you don't have to re-fetch and re-render everything) and the scroll offset. The common failure is relying only on Angular Router's built-in `scrollPositionRestoration`, which restores scroll but not data — if the component is destroyed and recreated, you're back to page 1 and scrolling to a position with no content there yet.

The robust approach: cache the list state (items array, page cursor, scroll offset) in a shared service (or component store) keyed by route, keep the component alive with `RouteReuseStrategy` if feasible, or manually restore on `ngOnInit`/constructor before the view paints.

```
import { Injectable, signal } from '@angular/core';

interface FeedState {
  items: Item[];
  page: number;
  hasMore: boolean;
  scrollY: number;
}

@Injectable({ providedIn: 'root' })
export class FeedStateCache {
  private cache = new Map<string, FeedState>();

  save(key: string, state: FeedState) {
    this.cache.set(key, state);
  }

  restore(key: string): FeedState | undefined {
    return this.cache.get(key);
  }

  clear(key: string) {
    this.cache.delete(key);
  }
}
```

```

@Component({ /* ... */ })
export class ArticleFeedComponent {
  private cache = inject(FeedStateCache);
  private router = inject(Router);
  private readonly cacheKey = 'article-feed';

  items = signal<Item[]>([]);
  page = signal(0);
  hasMore = signal(true);

  constructor() {
    const saved = this.cache.restore(this.cacheKey);
    if (saved) {
      this.items.set(saved.items);
      this.page.set(saved.page);
      this.hasMore.set(saved.hasMore);
      // Wait a frame so the DOM has rendered the restored items, then scroll
      requestAnimationFrame(() => window.scrollTo(0, saved.scrollY));
    } else {
      this.loadNextPage();
    }
  }

  ngOnDestroy() {
    this.cache.save(this.cacheKey, {
      items: this.items(),
      page: this.page(),
      hasMore: this.hasMore(),
      scrollY: window.scrollY,
    });
  }
}

```

Also disable Angular's default `scrollPositionRestoration: 'top'` for this route in `provideRouter`, or set it to `'disabled'` and handle scroll manually as above, since the two mechanisms can otherwise conflict. `requestAnimationFrame` (sometimes two frames, or a `setTimeout(0)`) ensures the DOM has painted the restored items before you set `scrollTop`, otherwise the page has no height yet to scroll into.

Interviewer intent: Tests awareness that scroll restoration is meaningless without data restoration, and that DOM paint timing matters when programmatically restoring scroll.

Scenario 4: Combining infinite scroll with filtering/sorting — resetting correctly

Situation: Users can filter the infinite-scrolling product list by category and change the sort order. After changing a filter, some old items from the previous filter briefly flash before the new ones load, and occasionally old and new pages get mixed together.

Question: How do you correctly reset infinite-scroll state when filters or sort order change, and how do you avoid interleaving stale in-flight requests with new ones?

Answer: The problem is twofold: (1) state (items, page, hasMore) isn't atomically reset when a filter changes, so the DOM briefly shows stale content, and (2) an in-flight request for the *old* filter can resolve *after* the reset and get appended to the *new* list. You need to both reset state synchronously and cancel/ignore stale in-flight requests — a "request generation" or `switchMap` on the filter stream handles this well.

```
import { toObservable } from '@angular/core/rxjs-interop';
import { switchMap, scan, startWith } from 'rxjs/operators';
import { combineLatest, Subject } from 'rxjs';

interface Filters { category: string; sort: 'price' | 'newest'; }

export class CatalogComponent {
  private catalogService = inject(CatalogService);

  filters = signal<Filters>({ category: 'all', sort: 'newest' });
  private loadMore$ = new Subject<void>();

  items = signal<Item[]>([]);
  page = signal(0);
  hasMore = signal(true);
  loading = signal(false);

  constructor() {
    const filters$ = toObservable(this.filters);

    filters$.pipe(
      // Any filter change resets state synchronously before firing a new fetch
      switchMap((filters) => {
        this.items.set([]);
        this.page.set(0);
        this.hasMore.set(true);
        this.loading.set(true);
        return this.catalogService.getPage(filters, 0);
      })
    ).subscribe((res) => this.applyPage(res, 0));

    this.loadMore$.pipe(
      switchMap(() => {
        if (!this.hasMore() || this.loading()) return EMPTY;
        this.loading.set(true);
        const nextPage = this.page() + 1;
        return this.catalogService.getPage(this.filters(), nextPage).pipe(
          map((res) => ({ res, nextPage }))
        );
      })
    ).subscribe(({ res, nextPage }) => this.applyPage(res, nextPage));
  }
}
```

```

private applyPage(res: PageResult, page: number) {
  this.items.update((cur) => (page === 0 ? res.items : [...cur, ...res.items]));
  this.hasMore.set(res.hasMore);
  this.page.set(page);
  this.loading.set(false);
}

onFilterChange(next: Filters) {
  this.filters.set(next);
}

onSentinelIntersect() {
  this.loadMore$.next();
}
}

```

The key insight: `switchMap` on the *filter* stream automatically cancels the previous filter's in-flight HTTP call when a new filter arrives (assuming the `HttpClient` observable is used directly, since unsubscribing cancels the underlying XHR/fetch). Combined with synchronously clearing `items` / `page` / `hasMore` at the start of the `switchMap` projector — not in the `.subscribe()` callback — you eliminate the flash-of-stale-content because the reset happens the instant the filter changes, not after the network round-trip.

Interviewer intent: Checks that the candidate understands the difference between resetting state eagerly vs. reactively, and that `switchMap` cancellation applies to HTTP observables specifically because unsubscription aborts the request.

Scenario 5: Memory growth from an ever-growing DOM list — needing windowing

Situation: A social media feed with infinite scroll works fine for the first few hundred posts, but after a user scrolls for 20 minutes, the tab's memory usage balloons and scrolling becomes janky. DevTools shows thousands of DOM nodes.

Question: How do you fix performance degradation from an unbounded infinite-scroll list, and what Angular tools help with virtualization/windowing?

Answer: The issue is that infinite scroll keeps *appending* to the DOM forever with no upper bound — every post's DOM nodes, bound event listeners, and change-detection subscriptions stay alive. The fix is virtual scrolling ("windowing"): only render the DOM nodes for items currently in (or near) the viewport, recycling nodes as the user scrolls, while keeping the underlying data model in memory (which is cheap) or itself paginated/evicted.

Angular CDK's `ScrollingModule` (`cdk-virtual-scroll-viewport`) is the standard tool, and it composes naturally with infinite-scroll fetching by listening to the viewport's scrolled-index to trigger the next page load instead of (or in addition to) an `IntersectionObserver` sentinel.

```

import { Component, inject, signal } from '@angular/core';
import { ScrollingModule, CdkVirtualScrollViewport } from '@angular/cdk/scrolling';

@Component({

```

```

selector: 'app-feed',
standalone: true,
imports: [ScrollingModule],
template: `
  <cdk-virtual-scroll-viewport itemSize="120" class="viewport"
(scrolledIndexChange)="onIndexChange($event)">
    <div *cdkVirtualFor="let post of posts(); trackBy: trackById" class="post-card">
      {{ post.text }}
    </div>
  </cdk-virtual-scroll-viewport>
`,
styles: [`.viewport { height: 100vh; }`],
})
export class FeedComponent {
  private feedService = inject(FeedService);
  posts = signal<Post[]>([]);
  page = signal(0);
  loading = signal(false);
  hasMore = signal(true);

  trackById = (_, number, post: Post) => post.id;

  onIndexChange(index: number) {
    const total = this.posts().length;
    // Fire the next page fetch when the user is within 10 items of the end
    if (index >= total - 10 && !this.loading() && this.hasMore()) {
      this.loadNextPage();
    }
  }

  private loadNextPage() {
    this.loading.set(true);
    this.feedService.getPage(this.page()).subscribe((res) => {
      this.posts.update((cur) => [...cur, ...res.items]);
      this.hasMore.set(res.hasMore);
      this.page.update((p) => p + 1);
      this.loading.set(false);
    });
  }
}

```

For truly unbounded feeds (hours of scrolling), you additionally want to *evict* old items from the data array once they're far enough behind the viewport (e.g., keep a sliding window of the last 500 items in memory, dropping the oldest as new ones arrive), since even non-rendered array entries with attached image blobs or large objects can add up. `cdk-virtual-scroll-viewport` handles DOM recycling; eviction of the underlying array is your own responsibility if the session is extremely long-lived.

Interviewer intent: Distinguishes candidates who know "infinite scroll" and "virtual scrolling" are complementary, not the same thing, and who understand CDK's virtual scroll API and index-based fetch triggers.

Scenario 6: Loading spinner and error-state UX at the end of a list

Situation: The infinite-scroll feed works, but when the API call to fetch the next page fails (e.g., flaky network), the UI just silently stops loading more content with no feedback, and the user assumes that's the end of the list.

Question: How do you design robust loading/error/end-of-list UX states for infinite scroll, and how do you let the user retry a failed page load?

Answer: You need a small state machine per "load more" attempt — `idle | loading | error | done` — surfaced distinctly in the template, with a retry action that re-triggers the same page fetch (not the next one, to avoid skipping data).

```
type LoadState = 'idle' | 'loading' | 'error' | 'done';

@Component({
  standalone: true,
  template: `
    @for (item of items(); track item.id) {
      <div class="item">{{ item.title }}</div>
    }

    <div #sentinel></div>

    @switch (loadState()) {
      @case ('loading') {
        <div class="spinner" role="status" aria-live="polite">Loading more...</div>
      }
      @case ('error') {
        <div class="error-banner" role="alert">
          Couldn't load more items.
          <button (click)="retry()">Retry</button>
        </div>
      }
      @case ('done') {
        <p class="end-marker">You've reached the end.</p>
      }
    }
  `,
})
export class FeedComponent {
  private feedService = inject(FeedService);
  items = signal<Item[]>([]);
  page = signal(0);
  loadState = signal<LoadState>('idle');

  loadNextPage() {
    if (this.loadState() === 'loading') return;
    this.loadState.set('loading');
    this.feedService.getPage(this.page()).subscribe({
      next: (res) => {
```

```

        this.items.update((cur) => [...cur, ...res.items]);
        this.page.update((p) => p + 1);
        this.loadState.set(res.hasMore ? 'idle' : 'done');
    },
    error: () => {
        this.loadState.set('error');
    },
  });
}

retry() {
  // Re-attempt the SAME page number, not page + 1, since the previous
  // attempt never succeeded and never incremented `page`.
  this.loadNextPage();
}
}

```

Important details: the observer should stop calling `loadNextPage()` while `loadState() === 'error'` (otherwise a failing fast-scroll spams retries) — gate the IntersectionObserver callback on `loadState() === 'idle'`. The explicit "done" state with an end-marker communicates to the user that scrolling further won't fetch anything, preventing confusion with a silently-stalled feed. `aria-live="polite"` and `role="alert"` ensure screen reader users get non-visual feedback on these transient states too.

Interviewer intent: Tests whether the candidate treats failure and completion as first-class UI states rather than afterthoughts, and whether retry logic correctly avoids skipping or duplicating a page.

Scenario 7: Accessibility concerns with infinite scroll

Situation: An accessibility audit flags the infinite-scroll product feed: screen reader users have no idea new content loaded, keyboard-only users can't easily reach the footer (which keeps moving as content loads), and the "sentinel" div is read aloud by the screen reader as empty content.

Question: What accessibility problems does infinite scroll typically introduce, and how do you address each one in Angular?

Answer: Infinite scroll breaks several implicit accessibility contracts: (1) content changes are not announced, (2) the page's logical end (footer, "contact us" links) becomes unreachable or keeps shifting, (3) focus can get lost when new DOM nodes are inserted, and (4) purely mouse/touch-driven scroll triggers exclude keyboard users who page through content differently.

Mitigations:

```

@Component({
  standalone: true,
  template: `
    <ul aria-label="Product results">
      @for (item of items(); track item.id) {
        <li>{{ item.name }}</li>
      }
    </ul>
  `
})

```

```

<!-- visually hidden live region announces new content to screen readers -->
<div class="sr-only" aria-live="polite" aria-atomic="true">
  {{ announcement() }}
</div>

<div #sentinel aria-hidden="true" style="height:1px;"></div>

<!-- Explicit, keyboard-accessible alternative to scroll-triggered loading -->
<button (click)="loadNextPage()" [disabled]="loading() || !hasMore()">
  {{ hasMore() ? 'Load more products' : 'No more products' }}
</button>

<!-- skip link so keyboard users can bypass the ever-growing list -->
<a href="#site-footer" class="skip-link">Skip product list to footer</a>
`
,
})
export class AccessibleFeedComponent {
  items = signal<Item[]>([]);
  loading = signal(false);
  hasMore = signal(true);
  announcement = signal('');

  loadNextPage() {
    /* ...fetch logic... */
    // After appending:
    this.announcement.set(`${this.items().length} products loaded.`);
  }
}

```

Key points: `aria-hidden="true"` on the sentinel prevents screen readers from announcing an empty, meaningless div. The `aria-live="polite"` region announces "N products loaded" after each batch without interrupting the user's current reading. Providing a visible, focusable "Load more" button as a *fallback trigger* (not just the IntersectionObserver) ensures keyboard-only and switch-device users aren't dependent on scroll gestures alone — many production apps keep both: the observer for mouse/touch users, and the button always available as a manual escape hatch. A skip link lets keyboard users jump past a list that could otherwise grow indefinitely between them and the footer, satisfying WCAG's "bypass blocks" criterion.

Interviewer intent: Evaluates whether the candidate thinks beyond visual/functional correctness to WCAG concerns — live regions, focus management, and providing non-scroll-dependent interaction paths.

Scenario 8: SEO/SSR implications of infinite scroll content

Situation: Marketing complains that only the first 20 blog posts of an infinite-scroll blog index are showing up in Google search results, even though hundreds of posts exist and the app uses Angular Universal (SSR).

Question: What are the SEO implications of infinite scroll, and how do you make deep content crawlable while keeping the UX of infinite scroll for real users?

Answer: Search crawlers generally don't scroll or fire `IntersectionObserver` callbacks (Googlebot renders pages but doesn't simulate user scroll interaction reliably for JS-triggered fetches), so any content loaded only via scroll-triggered XHR after the initial SSR paint is often invisible to indexing, and there's no unique URL per "page" of content for it to rank pages individually anyway.

The standard fix is to pair infinite scroll with **paginated URLs** underneath: each "page" of the feed also has a real, crawlable, SSR-rendered route (`/blog?page=2`), and you progressively enhance it into infinite scroll for JS-enabled users, using `rel="next" / rel="prev"` link tags (or a sitemap) so crawlers can discover subsequent pages via normal link-following rather than scroll events.

```
import { Component, inject, PLATFORM_ID } from '@angular/core';
import { isPlatformBrowser } from '@angular/common';
import { ActivatedRoute, Router } from '@angular/router';
import { Meta } from '@angular/platform-browser';

@Component({
  standalone: true,
  template: `
    @for (post of posts(); track post.id) {
      <article><h2>{{ post.title }}</h2></article>
    }
    <div #sentinel></div>

    <!-- Crawlable fallback pagination, hidden visually for JS users but present in DOM -->
    <nav class="pagination-fallback" aria-hidden="false">
      @if (prevPage()) { <a [routerLink]="[]" [queryParams]="{ page: prevPage() }">Previous</a> }
      @if (nextPage()) { <a [routerLink]="[]" [queryParams]="{ page: nextPage() }">Next</a> }
    </nav>
  `,
})
export class BlogIndexComponent {
  private platformId = inject(PLATFORM_ID);
  private route = inject(ActivatedRoute);
  private meta = inject(Meta);

  posts = signal<Post[]>([]);
  currentPage = signal(1);
  prevPage = computed(() => (this.currentPage() > 1 ? this.currentPage() - 1 : null));
  nextPage = computed(() => this.currentPage() + 1); // assume more; real hasMore check
  // omitted for brevity

  constructor() {
    const pageParam = Number(this.route.snapshot.queryParamMap.get('page')) || 1;
    this.currentPage.set(pageParam);
    this.loadPage(pageParam); // SSR-rendered on the server for this exact page

    // rel=next/prev hints for crawlers (supplements sitemap-based discovery)
    if (this.prevPage()) {
      this.meta.updateTag({ rel: 'prev', href: `/blog?page=${this.prevPage()}` } as any);
    }
  }
}
```

```

    // Only attach scroll-driven infinite loading in the browser, never during SSR
    if (isPlatformBrowser(this.platformId)) {
        this.setupInfiniteScroll();
    }
}

private setupInfiniteScroll() {
    // IntersectionObserver wiring here – appends subsequent pages client-side
    // on top of the page that was already SSR-rendered for this exact URL.
}

private loadPage(page: number) {
    // fetch and set posts() for the requested page
}
}

```

Additional measures: expose an XML sitemap listing every paginated URL so Google discovers deep pages without depending on any interaction at all; ensure each paginated URL SSR-renders its own unique content (not just page 1 for every URL) so `curl`/crawler fetches return real HTML; avoid `history.pushState`-only navigation for pages that should be independently indexable — use real router navigation with query params or path segments. The core principle: infinite scroll is a *client-side UX enhancement* layered on top of a fully link-based, SSR-crawlable pagination structure — never the only way to reach content.

Interviewer intent: Tests whether the candidate understands that scroll-triggered content is often invisible to SEO and knows the progressive-enhancement pattern of real paginated URLs plus SSR as the fix, not just "add more meta tags."

Scenario 9: Handling rapid scroll-to-bottom-and-back (thrashing)

Situation: On a fast trackpad, users can flick to the bottom of the loaded list, trigger a fetch, then immediately scroll back up before it resolves, and then flick down again — causing multiple overlapping fetches and a flickering "loading" indicator.

Question: How do you prevent request thrashing when a user rapidly scrolls back and forth near the load boundary?

Answer: Beyond the `loading` guard, you want to debounce the *intersection signal itself* so that transient, rapid enter/exit events near the boundary don't each independently attempt to trigger a fetch decision. Combine a short debounce on the raw `IntersectionObserver` stream with the `exhaustMap`-style in-flight guard from Scenario 2.

```

import { Subject } from 'rxjs';
import { debounceTime, filter, exhaustMap } from 'rxjs/operators';

export class FeedComponent {
    private intersect$ = new Subject<boolean>();

    constructor(private feedService: FeedService) {

```

```

    this.intersect$
      .pipe(
        debounceTime(150), // collapse rapid enter/exit flicker into one signal
        filter((isIntersecting) => isIntersecting && this.hasMore() && !this.loading())
      )
      .pipe(
        exhaustMap(() => this.fetchNextPage())
      )
      .subscribe();
  }

  private observer = new IntersectionObserver((entries) => {
    this.intersect$.next(entries[0].isIntersecting);
  });

  // ...fetchNextPage as in scenario 2
}

```

The `debounceTime(150)` absorbs the noisy sequence of true/false/true events that a fast flick produces (the sentinel entering and leaving the viewport several times within a couple hundred milliseconds), so only the settled intersection state triggers evaluation. This is a UX-quality fix layered on top of the correctness fix (`exhaustMap` + `loading` guard) — without the correctness fix, debouncing alone wouldn't prevent duplicate fetches if a request happens to be in flight when the debounced signal fires again.

Interviewer intent: Distinguishes between correctness guards (must-have) and UX-smoothing debounce (nice-to-have), and checks the candidate doesn't conflate the two as the same fix.

Scenario 10: Infinite scroll inside a modal/nested scrollable container

Situation: A "select supplier" modal contains its own infinite-scrolling list, but the `IntersectionObserver` never fires because by default it observes intersection relative to the browser viewport, not the modal's internal scroll container.

Question: Why does `IntersectionObserver` fail to trigger inside a scrollable modal, and how do you fix it?

Answer: `IntersectionObserver`'s `root` option defaults to `null`, meaning the browser viewport. If the scrollable ancestor is a `div` with `overflow: auto` (as in a modal), the sentinel might never leave the *browser* viewport even though it's off-screen within the modal's own scroll area, so the callback never fires as expected — or fires immediately and constantly because it's always "in the viewport" from the document's perspective. The fix is to explicitly set `root` to the modal's scrollable container element.

```

import { Component, ElementRef, ViewChild, effect } from '@angular/core';

@Component({
  standalone: true,
  template: `
    <div #scrollContainer class="modal-scroll-area">
      @for (supplier of suppliers(); track supplier.id) {
        <div class="row">{{ supplier.name }}</div>
      }
    </div>
  `
})

```

```

    }
    <div #sentinel></div>
  </div>
  `,
  styles: [`.modal-scroll-area { max-height: 400px; overflow-y: auto; }`],
})
export class SupplierModalComponent {
  scrollContainer = viewChild.required<ElementRef<HTMLDivElement>>('scrollContainer');
  sentinel = viewChild.required<ElementRef<HTMLDivElement>>('sentinel');

  suppliers = signal<Supplier[]>([]);

  constructor() {
    effect((onCleanup) => {
      const observer = new IntersectionObserver(
        (entries) => {
          if (entries[0].isIntersecting) this.loadNextPage();
        },
        {
          root: this.scrollContainer().nativeElement, // scoped to the modal, not the
document
          rootMargin: '100px',
          threshold: 0,
        }
      );
      observer.observe(this.sentinel().nativeElement);
      onCleanup(() => observer.disconnect());
    });
  }

  private loadNextPage() { /* ... */ }
}

```

A related gotcha: `root` must be a scrollable ancestor of the sentinel *and* actually have `overflow: auto/scroll` with a bounded height — if the container has `overflow: visible` or no explicit height, the observer either throws (in some browsers) or never fires because there's nothing to "scroll" against. Also, if the modal itself is conditionally rendered (`@if`), you must re-create the observer whenever the modal reopens, since the previous container element is destroyed — hooking into the `effect`'s automatic re-run on `viewChild` signal changes handles this naturally.

Interviewer intent: Verifies the candidate knows `IntersectionObserver`'s `root` option and doesn't assume default viewport-relative behavior always applies, a very common real-world bug in nested scroll UIs.

Scenario 11: Restoring exact scroll offset after a "jump to item" deep link

Situation: A user shares a link to a specific comment (`/thread/42?highlight=comment-981`) deep inside a long, infinite-scrolling comment thread. Opening the link should load enough pages to reach that comment and scroll it into view — not force the user to scroll manually from page 1.

Question: How do you implement "jump to and highlight a specific item" in an infinite-scroll list when that item may be many pages deep?

Answer: You need a mode that fetches sequentially (or via a dedicated "fetch up to and including item X" endpoint, if the API supports it) until the target item is present in the loaded set, then scrolls it into view and flags it visually — all before handing control back to normal scroll-triggered pagination.

```
@Component({ /* ... */ })
export class ThreadComponent {
  private commentService = inject(CommentService);
  private route = inject(ActivatedRoute);

  comments = signal<Comment[]>([]);
  page = signal(0);
  hasMore = signal(true);
  highlightId = signal<string | null>(null);

  constructor() {
    const target = this.route.snapshot.queryParamMap.get('highlight');
    this.highlightId.set(target);
    if (target) {
      this.loadUntilFound(target);
    } else {
      this.loadNextPage();
    }
  }

  private loadUntilFound(targetId: string, attempt = 0) {
    if (attempt > 50) return; // safety cap against infinite loop if item doesn't exist
    this.commentService.getPage(this.page()).subscribe((res) => {
      this.comments.update((cur) => [...cur, ...res.items]);
      this.page.update((p) => p + 1);
      this.hasMore.set(res.hasMore);

      const found = res.items.some((c) => c.id === targetId);
      if (found) {
        // wait for DOM render, then scroll the target into view
        requestAnimationFrame(() => {
          document.getElementById(`comment-${targetId}`)?.scrollIntoView({ block: 'center'
        });
      }
    });
    } else if (this.hasMore()) {
      this.loadUntilFound(targetId, attempt + 1);
    }
  }
}
```



```
// Template excerpt showing the highlight styling
// @for (comment of comments(); track comment.id) {
//   <div [id]='comment-' + comment.id"
//       [class.highlighted]='comment.id === highlightId()'>
//     {{ comment.text }}
//   </div>
// }
```

Tradeoffs: sequentially loading every page until the target is found is simple but wasteful for deeply nested items (e.g., page 40 of 40) — if the backend can support it, a better API is one that accepts a `?around=commentId` parameter and returns a window of items centered on that comment directly, avoiding N sequential round trips. The `attempt` cap guards against an infinite fetch loop if the target ID no longer exists (e.g., the comment was deleted). `scrollIntoView({ block: 'center' })` after `requestAnimationFrame` ensures the DOM has actually painted the newly appended batch before attempting to measure/scroll to it.

Interviewer intent: Tests whether the candidate can reason about a real product requirement (deep linking into paginated content) beyond the basic append-on-scroll pattern, and considers backend API design tradeoffs.

Scenario 12: Infinite scroll combined with pull-to-refresh at the top

Situation: A news feed app needs both infinite scroll (loading older articles at the bottom) and pull-to-refresh (loading newest articles at the top). Engineers worry about conflicting scroll-position math and duplicate/mixed content when both operations can happen close together.

Question: How do you architect state so that prepending new items at the top (refresh) and appending old items at the bottom (infinite scroll) don't corrupt each other or the scroll position?

Answer: Treat the list as having two independent "edges," each with its own loading/cursor state, but a single shared items array. Prepending requires *scroll anchoring*: naively unshifting content at the top scrolls the viewport visually because the browser doesn't compensate for content inserted above the current scroll position, so the user's read position visually jumps.

```
@Component({ /* ... */ })
export class NewsFeedComponent {
  items = signal<Article[]>([]);

  // Bottom-edge state (older articles / infinite scroll)
  oldestCursor = signal<string | null>(null);
  loadingOlder = signal(false);
  hasOlder = signal(true);

  // Top-edge state (newest articles / pull-to-refresh)
  newestCursor = signal<string | null>(null);
  loadingNewer = signal(false);

  constructor(private feedService: FeedService, private el: ElementRef) {}

  loadOlder() {
```

```

    if (this.loadingOlder() || !this.hasOlder()) return;
    this.loadingOlder.set(true);
    this.feedService.getOlderThan(this.oldestCursor()).subscribe((res) => {
        // Append at the bottom -- no scroll compensation needed, natural growth
        this.items.update((cur) => [...cur, ...res.items]);
        this.oldestCursor.set(res.nextCursor);
        this.hasOlder.set(res.hasMore);
        this.loadingOlder.set(false);
    });
}

refreshNewer() {
    if (this.loadingNewer()) return;
    this.loadingNewer.set(true);
    const container = this.el.nativeElement.querySelector('.scroll-area') as HTMLElement;
    const prevScrollHeight = container.scrollHeight;
    const prevScrollTop = container.scrollTop;

    this.feedService.getNewerThan(this.newestCursor()).subscribe((res) => {
        if (res.items.length) {
            this.items.update((cur) => [...res.items, ...cur]);
            this.newestCursor.set(res.items[0].id);

            // Scroll anchoring: after prepend, DOM grows above the current view,
            // so add the height difference back to scrollTop to keep the same
            // visual reading position instead of jumping to show the new items.
            requestAnimationFrame(() => {
                const newScrollHeight = container.scrollHeight;
                container.scrollTop = prevScrollTop + (newScrollHeight - prevScrollHeight);
            });
        }
        this.loadingNewer.set(false);
    });
}
}

```

Two independent cursors (`oldestCursor` / `newestCursor`) prevent the "load older" and "refresh newer" operations from fighting over a single "page number" concept, which breaks down once you're inserting at both ends of a growing list. The scroll-anchoring math (`prevScrollTop + (newScrollHeight - prevScrollHeight)`) is the standard technique for keeping visual position stable after prepending — without it, every pull-to-refresh would yank the user's scroll position to show the top, even if they were reading further down.

Interviewer intent: Assesses whether the candidate has hit the specific, non-obvious "scroll jump on prepend" bug in practice and knows the scroll-anchoring formula, a strong signal of real production experience.

Scenario 13: Testing infinite scroll behavior in unit/integration tests

Situation: The team's CI pipeline has no coverage for the infinite-scroll component because "IntersectionObserver doesn't exist in the test environment (jsdom)" and the previous developer just skipped the tests.

Question: How do you unit test a component whose pagination is triggered by IntersectionObserver, given that jsdom doesn't implement it?

Answer: Two viable strategies: (1) mock `IntersectionObserver` globally in the test setup and manually invoke the captured callback to simulate intersection, or (2) refactor the component so the "trigger" is a separate, injectable/testable unit (e.g., an Angular CDK-style directive or a plain function you can call directly), keeping the actual browser API as a thin adapter. Strategy 1 is usually simplest and keeps the component code untouched.

```
// test-setup: mock IntersectionObserver so components can be instantiated in jsdom
class MockIntersectionObserver {
  static instances: MockIntersectionObserver[] = [];
  callback: IntersectionObserverCallback;
  constructor(callback: IntersectionObserverCallback) {
    this.callback = callback;
    MockIntersectionObserver.instances.push(this);
  }
  observe = jest.fn();
  unobserve = jest.fn();
  disconnect = jest.fn();

  // Test helper to simulate the sentinel becoming visible
  trigger(isIntersecting: boolean) {
    this.callback(
      [{ isIntersecting } as IntersectionObserverEntry],
      this as unknown as IntersectionObserver
    );
  }
}

beforeEach(() => {
  MockIntersectionObserver.instances = [];
  (globalThis as any).IntersectionObserver = MockIntersectionObserver;
});
```

```
it('loads the next page when the sentinel intersects', fakeAsync(() => {
  const feedServiceSpy = { getPage: jest.fn().mockReturnValue(of({ items: [{ id: 1 }],
    hasMore: true })) };
  TestBed.configureTestingModule({
    imports: [ProductListComponent],
    providers: [{ provide: ProductService, useValue: feedServiceSpy }],
  });

  const fixture = TestBed.createComponent(ProductListComponent);
```

```

fixture.detectChanges();
tick();

const observerInstance = MockIntersectionObserver.instances[0];
observerInstance.trigger(true); // simulate sentinel entering viewport
tick();
fixture.detectChanges();

expect(feedServiceSpy.getPage).toHaveBeenCalled();
expect(fixture.componentInstance.items().length).toBe(1);
});

it('does not fire duplicate fetches for rapid re-intersections while loading', fakeAsync(()
=> {
  // ...configure a slow observable with delay(), trigger(true) twice quickly,
  // and assert getPage was called only once until the first resolves.
}));

```

The mock captures the callback the component registered so the test can call `.trigger(true/false)` directly, fully decoupling the test from real browser scroll geometry. This lets you assert not just "loads on intersect" but the harder cases from earlier scenarios — no duplicate fetch while loading, no fetch after `hasMore` is false, error state transitions — all deterministically, without flaky timing-based scroll simulation.

Interviewer intent: Tests practical knowledge of testing browser-API-dependent components, and whether the candidate can design tests for the subtle race-condition and lifecycle bugs discussed elsewhere in this chapter, not just the happy path.

Scenario 14: Multiple simultaneous infinite-scroll lists on one page (tabbed interface)

Situation: A dashboard has three tabs, each showing its own infinite-scrolling list (Orders, Invoices, Shipments). Switching tabs quickly causes the wrong tab's spinner to show, or a fetch from a previously active tab to land in the wrong tab's list after switching back.

Question: How do you isolate infinite-scroll state per tab so that state and in-flight requests from one tab never leak into another?

Answer: The fix is to give each tab its own fully independent instance of state and observer — never a single shared signal/service instance reused across tabs. If tabs are implemented with `@if / @switch` that destroys and recreates components, this is naturally isolated as long as state lives in the *component*, not a singleton service. If tabs are kept alive (e.g., wrapped in `[hidden]` to preserve state across switches, a common UX choice so users don't lose scroll position when tabbing away and back), each tab component still needs its own instance-scoped service via a component-level provider, not `providedIn: 'root'`.

```

// Per-tab feed component: each tab gets its own instance because
// FeedStateService is provided at the COMPONENT level, not root.
@Injectable() // note: no providedIn: 'root' -- must be provided per-component
export class FeedStateService {
  items = signal<Item[]>([]);
  page = signal(0);

```

```

loading = signal(false);
hasMore = signal(true);

constructor(private http: HttpClient, private endpoint: string) {}
}

@Component({
  selector: 'app-orders-tab',
  standalone: true,
  providers: [
    { provide: FeedStateService, useFactory: () => new FeedStateService(inject(HttpClient),
'/api/orders') },
  ],
  template: `<app-generic-feed [state]="state" />`,
})
export class OrdersTabComponent {
  state = inject(FeedStateService);
}

```

```

@Component({
  selector: 'app-tabs',
  standalone: true,
  imports: [OrdersTabComponent, InvoicesTabComponent, ShipmentsTabComponent],
  template: `
    <nav>
      @for (tab of tabs; track tab) {
        <button (click)="active.set(tab)">{{ tab }}</button>
      }
    </nav>

    <!-- [hidden] keeps components alive (preserving scroll + data) rather than
      destroying/recreating them on every tab switch -->
    <div [hidden]="active() !== 'orders'"><app-orders-tab /></div>
    <div [hidden]="active() !== 'invoices'"><app-invoices-tab /></div>
    <div [hidden]="active() !== 'shipments'"><app-shipments-tab /></div>
  `,
})
export class TabsComponent {
  tabs = ['orders', 'invoices', 'shipments'] as const;
  active = signal<typeof this.tabs[number]>('orders');
}

```

Each tab's `FeedStateService` is a distinct instance (via `useFactory` + component-level `providers`), so its `items` / `page` / `loading` signals and its `IntersectionObserver` are fully isolated — a slow response for the Invoices tab can never touch the Orders tab's signals because they're different objects entirely. Using `[hidden]` instead of `@if` keeps each tab's DOM, scroll position, and `IntersectionObserver` alive across switches (so revisiting a tab doesn't refetch from scratch), at the cost of keeping all three tabs' observers registered simultaneously — an acceptable tradeoff for a small, fixed number of tabs, but worth reconsidering if there could be many.

Interviewer intent: Tests understanding of Angular DI instance scoping (`providedIn: 'root'` singleton pitfalls) and the `@if` vs `[hidden]` tradeoff for preserving component/scroll state.

Scenario 15: Server returns overlapping/shifted data because items were inserted mid-scroll

Situation: In a live-updating admin table with infinite scroll, new records are constantly being inserted by other users. Customers report seeing the same row twice, or a row missing entirely, purely from scrolling through offset-based pages while data is changing underneath them.

Question: Why does offset-based (`LIMIT/OFFSET`) pagination break under infinite scroll when the underlying dataset is being mutated concurrently, and how do you fix it on the client and API contract?

Answer: Offset pagination assumes a stable ordering across requests. If page 1 fetches rows 0–19 and then a new row is inserted at the front before page 2 fetches rows 20–39, every row shifts by one — row 20 (now what was row 19) gets shown twice, and the item that was at true offset 20 is skipped. This is a fundamental flaw of `OFFSET`, not something you can patch purely client-side; the real fix is switching the API to **cursor-based pagination** (keyset pagination), where each page request says "give me items after cursor X" using a stable, monotonic field (e.g., `created_at` + `id` tie-breaker) rather than a numeric offset that shifts as rows are inserted/deleted.

```
// Cursor-based fetch contract -- immune to insertions shifting offsets
interface PageResult<T> {
  items: T[];
  nextCursor: string | null; // opaque cursor, e.g. base64 of (createdAt, id)
  hasMore: boolean;
}

export class LiveTableService {
  private http = inject(HttpClient);

  getPage(cursor: string | null): Observable<PageResult<Row>> {
    const params = cursor ? new HttpParams().set('cursor', cursor) : new HttpParams();
    return this.http.get<PageResult<Row>>('/api/rows', { params });
  }
}
```

```
export class LiveTableComponent {
  private liveTableService = inject(LiveTableService);
  rows = signal<Row[]>([]);
  cursor = signal<string | null>(null);
  hasMore = signal(true);

  loadNextPage() {
    this.liveTableService.getPage(this.cursor()).subscribe((res) => {
      // Defensive dedupe: even with cursors, guard against client-side re-fetch bugs
      this.rows.update((cur) => {
        const seen = new Set(cur.map((r) => r.id));
        return [...cur, ...res.items.filter((r) => !seen.has(r.id))];
      });
      this.cursor.set(res.nextCursor);
      this.hasMore.set(res.hasMore);
    });
  }
}
```

```
}  
}
```

Cursor pagination is stable under concurrent inserts/deletes because "after this specific row" doesn't shift meaning the way "after position 20" does — an insertion anywhere in the dataset simply doesn't change what "after row X" refers to. If you can't change the API (e.g., third-party), the client-side mitigation is defensive deduplication by unique id (shown above) plus accepting that occasional gaps are a known limitation to surface (e.g., a "new items available, refresh to see" banner) rather than silently corrupting the list.

Interviewer intent: Tests systems-level understanding that this is fundamentally a pagination-strategy/API-design problem, not something client code alone can fully solve, and knowledge of cursor vs. offset pagination tradeoffs.

Scenario 16: Infinite scroll with `@defer` for below-the-fold sections

Situation: A long marketing page has an infinite-scrolling "related articles" widget far below the fold. The team wants to avoid shipping and evaluating that component's JS/logic at all until the user actually scrolls near it, to improve initial bundle evaluation cost and Core Web Vitals.

Question: How can Angular's `@defer` block be combined with infinite scroll to lazy-load both the code and the data for a below-the-fold feed?

Answer: `@defer (on viewport)` is purpose-built for this: it defers loading the component's JS chunk until the placeholder scrolls near the viewport, using an `IntersectionObserver` internally. This is complementary to, not a replacement for, the infinite-scroll pagination *within* that component — `@defer` solves "don't load this widget's code/module until needed," while your own `IntersectionObserver`/CDK logic solves "keep loading more pages once the user is scrolling within the widget."

```
@Component({  
  standalone: true,  
  template: `  
    <article>{{ mainContent }}</article>  
  
    @defer (on viewport; prefetch on idle) {  
      <app-related-articles-feed />  
    } @placeholder (minimum 100ms) {  
      <div class="feed-placeholder" style="height: 400px;"></div>  
    } @loading (minimum 200ms) {  
      <div class="spinner">Loading related articles...</div>  
    } @error {  
      <p>Couldn't load related articles.</p>  
    }  
  `,  
})  
export class ArticlePageComponent {  
  mainContent = '...';  
}
```

```
// RelatedArticlesFeedComponent itself implements its own scroll-triggered
```

```
// infinite scroll internally once it IS loaded and rendered -- unrelated
// to the outer @defer, which only gated the initial code+render.
@Component({
  selector: 'app-related-articles-feed',
  standalone: true,
  template: `
    @for (article of articles(); track article.id) {
      <a [routerLink]="['/blog', article.slug]">{{ article.title }}</a>
    }
    <div #sentinel></div>
  `,
})
export class RelatedArticlesFeedComponent {
  // ...IntersectionObserver-driven pagination as in Scenario 1
}
```

`prefetch on idle` lets Angular fetch the *component's JS chunk* during idle browser time even before it's near the viewport, so that when `on viewport` finally fires (the placeholder nears the screen), the code is likely already downloaded and only rendering/data-fetching remains — reducing perceived latency versus fetching the chunk cold. The `@placeholder`, `@loading`, and `@error` sub-blocks give you the same "minimum display time" and failure-state ergonomics `@defer` provides natively, layered on top of the feed's own internal infinite-scroll loading/error states from Scenario 6 — the two loading indicators serve different scopes (module load vs. data page load) and shouldn't be conflated into one spinner.

Interviewer intent: Confirms the candidate knows `@defer`'s viewport trigger is about deferred *hydration/code-loading*, distinct from data pagination, and can layer the two correctly rather than treating `@defer` as a full infinite-scroll replacement.

Scenario 17: Handling window resize / zoom changing what counts as "near the bottom"

Situation: On a very tall, wide monitor, testers notice the infinite-scroll feed fetches many pages almost immediately on load because a huge viewport means several pages' worth of the sentinel-adjacent area are "close enough" to trigger loads back-to-back, faster than the UX intends.

Question: How do you make infinite-scroll trigger distance robust across different viewport sizes and zoom levels, avoiding both "loads too eagerly on huge screens" and "never triggers on tiny screens"?

Answer: A fixed pixel `rootMargin` (e.g., `'200px'`) behaves inconsistently across viewport sizes: on a 4K monitor showing 3 pages' worth of content at once, 200px might barely matter and trigger loads back to back as each newly-appended page still leaves the sentinel near-visible; on a small mobile viewport, the same 200px might be a much larger relative buffer. Rather than a fixed pixel margin, consider (a) capping how many *consecutive* auto-triggered fetches can happen without a user scroll gesture in between, or (b) sizing `rootMargin` relative to viewport height, or (c) simply accepting eager pre-fetch but bounding it with a stricter `hasMore`/rate-limit rather than fighting viewport geometry directly.

```
export class FeedComponent {
  private consecutiveAutoLoads = signal(0);
  private readonly MAX_CONSECUTIVE_AUTO_LOADS = 3;
  private lastUserScrollTime = 0;
```



```

constructor() {
  window.addEventListener('wheel', () => (this.lastUserScrollTime = Date.now()), {
    passive: true });
  window.addEventListener('touchmove', () => (this.lastUserScrollTime = Date.now()), {
    passive: true });

  effect((onCleanup) => {
    const vh = window.innerHeight;
    const observer = new IntersectionObserver(
      (entries) => {
        if (entries[0].isIntersecting) this.tryLoadNextPage();
      },
      {
        // Scale the pre-fetch margin relative to viewport height instead of
        // a fixed pixel value, so large viewports don't over-trigger.
        rootMargin: `${Math.round(vh * 0.5)}px`,
        threshold: 0,
      }
    );
    observer.observe(this.sentinelEl());
    onCleanup(() => observer.disconnect());
  });
}

private tryLoadNextPage() {
  const timeSinceUserScroll = Date.now() - this.lastUserScrollTime;
  const wasRecentUserGesture = timeSinceUserScroll < 1500;

  if (!wasRecentUserGesture && this.consecutiveAutoLoads() >=
this.MAX_CONSECUTIVE_AUTO_LOADS) {
    // Stop auto-triggering after N loads with no real user scroll input --
    // likely means the viewport is just huge, not that the user wants more.
    return;
  }

  this.consecutiveAutoLoads.update((n) => (wasRecentUserGesture ? 0 : n + 1));
  this.loadNextPage();
}
}

```

This is a nuanced UX-tuning problem more than a pure correctness bug: relative `rootMargin` (percentage of `innerHeight`) scales the pre-fetch distance sensibly across devices, while the "consecutive auto-load without a real scroll gesture" cap prevents a degenerate case where a component keeps eagerly fetching pages just because the sentinel technically stays within a large margin on a huge display, decoupled from any actual user intent to keep scrolling. Recomputing `rootMargin` on resize (re-running the `effect` when a `windowSize` signal changes) keeps it correct across zoom/orientation changes too.

Interviewer intent: Checks whether the candidate has thought past the "happy path" `IntersectionObserver` setup into real device/viewport variability, a sign of production hardening experience.

Scenario 18: Infinite scroll list where items themselves can change height dynamically

Situation: A chat-style feed loads more messages as the user scrolls up, but messages contain variable-height content (images, expandable code blocks). Occasionally the scroll position jumps unexpectedly as earlier images finish loading and change the layout above the viewport.

Question: How do you prevent scroll-position jumps caused by dynamic height changes in already-rendered infinite-scroll content (e.g., images loading asynchronously above the current viewport)?

Answer: This is the classic "layout shift above the fold" problem, distinct from the prepend-jump in Scenario 12 but solved with a related technique: you must reserve space for content whose final size isn't known yet, and/or apply scroll compensation whenever a `ResizeObserver` reports a height change for an element positioned above the current scroll position.

```
import { Component, ElementRef, ViewChild, effect } from '@angular/core';

@Component({
  standalone: true,
  template: `
    <div #scrollArea class="chat-scroll">
      @for (msg of messages(); track msg.id) {
        <div class="message" [attr.data-msg-id]="msg.id">
          @if (msg.imageUrl) {
            <!-- Reserve layout space via explicit dimensions to minimize shift -->
            <img [src]="msg.imageUrl" width="320" height="240" loading="lazy" />
          }
          <p>{{ msg.text }}</p>
        </div>
      }
    </div>
  `,
})
export class ChatFeedComponent {
  scrollArea = ViewChild<ElementRef<HTMLDivElement>>>('scrollArea');
  messages = signal<Message[]>([]);

  constructor() {
    effect((onCleanup) => {
      const container = this.scrollArea().nativeElement;

      // Watch every message element for size changes (e.g., image finishes loading)
      const resizeObserver = new ResizeObserver((entries) => {
        for (const entry of entries) {
          const el = entry.target as HTMLElement;
          const rect = el.getBoundingClientRect();
          const containerRect = container.getBoundingClientRect();

          // Only compensate if the resized element is ABOVE the visible viewport --
          // resizes below or within view don't need scroll adjustment.
          if (rect.bottom < containerRect.top) {
```

```

        const delta = entry.contentRect.height - (Number(e1.dataset['prevHeight']) ||
entry.contentRect.height);
        if (delta !== 0) {
            container.scrollTop += delta;
        }
    }
    e1.dataset['prevHeight'] = String(entry.contentRect.height);
}
});

const observeAll = () => {
    container.querySelectorAll('.message').forEach((e1) => resizeObserver.observe(e1));
};
observeAll();

onCleanup(() => resizeObserver.disconnect());
});
}
}

```

Explicit `width/height` attributes on `<img>` (matching the eventual rendered size) are the first line of defense — the browser reserves the correct box before the image loads, so there's no layout shift to compensate for at all, which is both simpler and better for Cumulative Layout Shift (CLS) scoring. The `ResizeObserver`-based compensation is a second line of defense for content whose final size genuinely can't be known upfront (e.g., collapsible code blocks, embeds); it only adjusts `scrollTop` when the resized element sits above the current viewport, since resizes below or within the visible area don't need any compensation and adjusting for them would itself cause an unwanted jump.

Interviewer intent: Tests whether the candidate distinguishes CLS-style layout shift from the prepend-scroll-jump problem, and knows both the preventive (reserved dimensions) and reactive (`ResizeObserver` compensation) techniques.

Scenario 19: Choosing between `IntersectionObserver` and a scroll event listener

Situation: A junior developer on the team implemented infinite scroll using a `(scroll)` event listener on the container with manual `scrollTop / scrollHeight / clientHeight` math, and asks why the tech lead insists on switching to `IntersectionObserver` instead, since "it works fine."

Question: What are the concrete advantages of `IntersectionObserver` over a scroll event listener for implementing infinite scroll, and are there any legitimate cases to still use scroll events?

Answer: A `(scroll)` listener fires synchronously and extremely frequently (potentially every few milliseconds during a fling-scroll), forcing you to manually throttle/debounce it and to read layout properties (`scrollTop`, `scrollHeight`, `clientHeight`) that can trigger forced synchronous layout reflows if read at the wrong time, directly on the main thread, competing with the browser's own scroll handling and risking jank.

`IntersectionObserver` runs asynchronously off the main scroll-handling critical path, is throttled/batched by the browser itself, and expresses the *intent* ("tell me when this element becomes visible") declaratively rather than via manual arithmetic that's easy to get subtly wrong (e.g., off-by-a-few-pixels threshold bugs, forgetting

to account for `clientHeight` vs `offsetHeight`, or not handling elastic/overscroll on iOS Safari where `scrollTop` can briefly go negative).

```
// Scroll-event approach (legacy, more error-prone, higher main-thread cost)
@HostListener('scroll', ['$event'])
onScroll(event: Event) {
  const el = event.target as HTMLElement;
  const threshold = 200;
  // Forces layout read on every scroll tick; needs manual throttling
  if (el.scrollHeight - el.scrollTop - el.clientHeight < threshold) {
    this.loadNextPage();
  }
}
```

```
// IntersectionObserver approach (preferred): declarative, async, browser-optimized
constructor() {
  effect((onCleanup) => {
    const observer = new IntersectionObserver(
      (entries) => entries[0].isIntersecting && this.loadNextPage(),
      { rootMargin: '200px' }
    );
    observer.observe(this.sentinel().nativeElement);
    onCleanup(() => observer.disconnect());
  });
}
```

Legitimate remaining uses of scroll events: when you need the *exact, continuous* scroll offset for something IntersectionObserver can't express, such as a parallax effect, a "scroll progress" indicator, or precise virtual-scroll windowing math (though CDK's virtual scroll already handles that internally) — i.e., cases needing a continuous value, not a boolean "is this visible" signal. For plain infinite-scroll triggering, IntersectionObserver is strictly better: less code, no manual throttling, no main-thread reflow forcing, and better battery/performance characteristics on mobile.

Interviewer intent: Tests whether the candidate can articulate *why* IntersectionObserver is preferred, not just that it's the "modern" choice, including the reflow/performance argument and honest exceptions where scroll events are still appropriate.

Scenario 20: Infinite scroll list needs to support drag-and-drop reordering of loaded items

Situation: In an admin task board, users can drag-and-drop reorder items within an infinite-scrolling list. QA finds that after scrolling to load more pages and then dragging an item, the reordered position resets or duplicates once another page loads.

Question: How do you make drag-and-drop reordering coexist safely with an infinite-scroll list whose underlying array keeps growing via appended pages?

Answer: The bug is almost always that "load next page" appends using stale array references or index-based assumptions that break once the user has already mutated order via drag-and-drop — e.g., appending with `[...originalOrderArray, ...newItems]` from a closure that captured an older, pre-reorder version of `items()`, or an index-based CDK drag-drop `moveItemInArray` operating on a copy that then gets clobbered by the next page's `.update()` call using an outdated snapshot semantics. The fix: always mutate through the signal's `.update()` with a callback receiving the *current* value (never close over a captured array), and keep drag-and-drop's reordering and infinite-scroll's appending as separate operations on the same signal, never on parallel copies.

```
import { CdkDragDrop, moveItemInArray, DragDropModule } from '@angular/cdk/drag-drop';

@Component({
  standalone: true,
  imports: [DragDropModule],
  template: `
    <div cdkDropList (cdkDropListDropped)="onDrop($event)">
      @for (task of tasks(); track task.id) {
        <div cdkDrag class="task-card">{{ task.title }}</div>
      }
    </div>
    <div #sentinel></div>
  `,
})
export class TaskBoardComponent {
  private taskService = inject(TaskService);
  tasks = signal<Task[]>([]);
  page = signal(0);
  loading = signal(false);

  onDrop(event: CdkDragDrop<Task[]>) {
    // Always derive the new order from the CURRENT signal value via .update(),
    // never from a variable captured earlier in the component's lifetime.
    this.tasks.update((current) => {
      const copy = [...current];
      moveItemInArray(copy, event.previousIndex, event.currentIndex);
      return copy;
    });

    // Persist the reorder so a later page-load (which reads server order)
    // doesn't silently undo it -- e.g., patch a `sortIndex` field per item.
    this.taskService.persistOrder(this.tasks()).subscribe();
  }

  loadNextPage() {
    if (this.loading()) return;
    this.loading.set(true);
    this.taskService.getPage(this.page()).subscribe((res) => {
      // Appending also goes through .update() with the CURRENT value,
      // so it composes correctly regardless of any prior drag-and-drop reorder.
      this.tasks.update((current) => {
        const existingIds = new Set(current.map((t) => t.id));
        const fresh = res.items.filter((t) => !existingIds.has(t.id));
      });
    });
  }
}
```

```

        return [...current, ...fresh];
    });
    this.page.update((p) => p + 1);
    this.loading.set(false);
  });
}
}

```

Two disciplines make this safe: first, both mutation paths (drag-and-drop reorder and page-append) go through `signal.update(currentValue => ...)`, which Angular guarantees receives the latest value at the time it runs, eliminating the stale-closure class of bugs entirely. Second, reordering is *persisted* immediately (`persistOrder`) so that if the server is later asked for "page 2," it returns items consistent with the user's chosen order rather than the original order, preventing the appearance of a reorder being "undone" by subsequent pagination. If the backend can't support per-user custom ordering, an alternative is to treat drag-and-drop reordering as purely a client-side/local-session concern and explicitly warn users that further scrolling loads more items in server order beneath their reordered section.

Interviewer intent: Tests whether the candidate understands why closures over signal values are dangerous for concurrent mutation paths, and can reason about product-level consistency (persisted order vs. transient client-only reorder) beyond just the Angular mechanics.

Quick Revision Cheat Sheet

- Use a sentinel element + `IntersectionObserver` (with `rootMargin` for pre-fetch) instead of raw `(scroll)` event math — it's async, browser-throttled, and avoids forced reflows.
- Always guard fetches synchronously with a `loading` signal set *before* the request starts, and prefer `RxJS exhaustMap` over `switchMap` / `mergeMap` / `concatMap` for "ignore triggers while a page load is in flight."
- Defensively dedupe appended items by unique id even when using `exhaustMap` — belt-and-suspenders against server-side overlap or edge-case races.
- Preserving scroll position on back-navigation requires restoring *both* the loaded data and the scroll offset (via a cache keyed by route), timed with `requestAnimationFrame` so the DOM has painted before you set `scrollTop`.
- When filters/sort change, reset `items` / `page` / `hasMore` synchronously inside the filter stream's projector (not after the response resolves), and let `switchMap` cancel stale in-flight requests tied to the old filter.
- Infinite scroll and virtual scrolling (CDK `cdk-virtual-scroll-viewport`) solve different problems — combine them for feeds that grow unboundedly, and consider evicting old data from memory for very long sessions.
- Treat `loading` | `error` | `done` as explicit, visible UI states with retry logic that re-fetches the *same* failed page, not `page + 1`.
- Accessibility requires `aria-live` announcements for newly loaded content, `aria-hidden` on the sentinel, a keyboard-reachable manual "load more" fallback, and a skip link past an unbounded list.
- Infinite scroll content loaded only via client-side scroll triggers is often invisible to search crawlers — pair it with real, SSR-rendered, crawlable paginated URLs (progressive enhancement), not scroll-only loading.
- Set `IntersectionObserver`'s `root` explicitly to a scrollable ancestor (e.g., a modal's inner container) — the default viewport root won't detect intersection inside nested scroll areas.

- Prepending items (pull-to-refresh, "load newer") requires scroll anchoring (`scrollTop += newScrollHeight - prevScrollHeight`) to avoid visually yanking the user's reading position.
- Use cursor/keyset pagination instead of offset/ `LIMIT-OFFSET` for any dataset that mutates concurrently with scrolling — offset pagination fundamentally breaks under concurrent inserts/deletes.
- Mock `IntersectionObserver` in tests to directly invoke its captured callback, letting you deterministically test intersection, race-condition guards, and error states without relying on real scroll geometry.
- Scope per-tab or per-instance infinite-scroll state via component-level DI providers (not `providedIn: 'root'` singletons) to prevent cross-tab/cross-instance state leakage.
- Layer `@defer (on viewport)` (code/module lazy-loading) on top of, not instead of, a feed's own internal data-pagination logic — they solve different loading concerns.
- Scale `rootMargin` relative to viewport height and consider capping consecutive auto-triggered loads to avoid over-eager fetching on very large screens.
- Reserve explicit image/media dimensions to prevent layout shift above the viewport, and use `ResizeObserver`-based `scrollTop` compensation as a fallback for content whose size can't be known upfront.
- Always mutate paginated signal state via `.update(current => ...)` rather than closures over previously captured arrays, especially when combining infinite-scroll appending with other mutations like drag-and-drop reordering.

Created By - Durgesh Singh